iroh 운영 및 모니터링 가이드

개요

이 문서는 iroh 기반 애플리케이션을 프로덕션 환경에서 안정적으로 운영하고 모니터링하기 위한 포괄적인 가이드를 제공합니다. iroh의 메트릭스 시스템, 설정 관리, 트러블슈팅, 그리고 성능 최적화 방법을 다룹니다.

1. 메트릭스 시스템 분석

1.1 iroh 메트릭스 아키텍처

iroh는 `iroh-metrics` 크레이트를 기반으로 한 포괄적인 메트릭스 시스템을 제공합니다. 메트릭스는 계층적 구조로 조직되어 있으며, 각 컴포넌트별로 세분화된 성능 지표를 수집합니다.

메트릭스 계층 구조

```
EndpointMetrics {
magicsock: MagicsockMetrics, // 네트워킹 계층 메트릭스
net_report: NetReportMetrics, // 네트워크 상태 보고 메트릭스
portmapper: PortmapMetrics, // 포트 매핑 메트릭스 (non-WASM)
}
```

1.2 주요 메트릭스 카테고리

MagicSock 메트릭스

연결 관련 메트릭스:

- `num_direct_conns_added` : 직접 연결 생성 횟수
- `num_direct_conns_removed` : 직접 연결 해제 횟수
- `num_relay_conns_added` : 릴레이 연결 생성 횟수
- `num_relay_conns_removed` : 릴레이 연결 해제 횟수
- `connection_handshake_success` : 성공한 핸드셰이크 횟수
- `connection_became_direct` : 릴레이에서 직접 연결로 전환된 횟수

데이터 전송 메트릭스:

- `send_data` : 전송된 데이터 패킷 수
- `recv_data_ipv4/ipv6` : IPv4/IPv6로 수신된 데이터 패킷 수
- `recv_data_relay` : 릴레이를 통해 수신된 데이터 패킷 수
- `recv_datagrams` : 수신된 QUIC 데이터그램 수
- `recv_gro_datagrams` : GRO(Generic Receive Offload)로 수신된 데이터그램 수

Discovery 메트릭스:

- `send_disco_udp/relay` : UDP/릴레이를 통한 discovery 패킷 전송
- `recv_disco_ping/pong` : discovery ping/pong 메시지 수신
- `recv_disco_call_me_maybe` : “call me maybe” 메시지 수신
- `recv_disco_bad_key/parse` : 잘못된 키 또는 파싱 오류

네트워크 상태 메트릭스:

- `update_direct_addrs` : 직접 주소 업데이트 횟수
- `relay_home_change` : 홈 릴레이 서버 변경 횟수
- `nodes_contacted` : 접촉 시도한 노드 수
- `nodes_contacted_directly` : 직접 접촉 성공한 노드 수

Net Report 메트릭스

- `reports` : 실행된 네트워크 보고서 총 수
- `reports_full` : 실행된 전체 네트워크 보고서 수

Relay Server 메트릭스

패킷 처리 메트릭스:

- `bytes_sent/recv` : 전송/수신된 바이트 수
- `send_packets_sent/recv/dropped` : 일반 패킷 전송/수신/드롭 수
- `disco_packets_sent/recv/dropped` : discovery 패킷 전송/수신/드롭 수
- `other_packets_sent/recv/dropped` : 기타 패킷 전송/수신/드롭 수

연결 관리 메트릭스:

- `accepts` : 수락된 연결 수
- `disconnects` : 연결 해제 수
- `unique_client_keys` : 고유 클라이언트 키 수

Rate Limiting 메트릭스:

- `bytes_rx_ratelimited_total` : Rate limit된 수신 바이트 수
- `conns_rx_ratelimited_total` : Rate limit된 연결 수

1.3 메트릭스 수집 및 접근

기본 메트릭스 접근

```
use iroh::Endpoint;


// Endpoint 생성
```

```rust
let endpoint = Endpoint::builder().bind().await?;


// 메트릭스 접근
let metrics = endpoint.metrics();


// 개별 메트릭스 값 확인
println!("수신된 데이터그램: {}", metrics.magicsock.recv_datagrams.get());
println!("직접 연결 수: {}", metrics.magicsock.num_direct_conns_added.get());
```

메트릭스 수집 및 내보내기

```rust
use std::collections::BTreeMap;
use iroh_metrics::{MetricsGroup, MetricValue, MetricsGroupSet};


// 모든 메트릭스를 맵으로 수집
let metrics_map: BTreeMap<String, MetricValue> = endpoint
.metrics()
.iter()
.map(|(group, metric)| {
let name = [group, metric.name()].join(":");
(name, metric.value())
})
.collect();
```

```rust
// 특정 메트릭스 확인
assert_eq!(metrics_map["magicsock:recv_datagrams"], MetricValue::Counter(0));
```

Prometheus/OpenMetrics 형식으로 내보내기

```rust
use iroh_metrics::{Registry, MetricsSource};

// 레지스트리 생성 및 메트릭스 등록
let mut registry = Registry::default();
registry.register_all(endpoint.metrics());

// OpenMetrics 형식으로 인코딩
let metrics_text = registry.encode_openmetrics_to_string()?;
println!("{}", metrics_text);
```

1.4 메트릭스 서버 설정

HTTP 메트릭스 서버 시작

```rust
use std::sync::{Arc, RwLock};
use iroh_metrics::{Registry, service::start_metrics_server};

// 공유 레지스트리 생성
let registry = Arc::new(RwLock::new(Registry::default()));
```

```rust
// 메트릭스 서버 시작
let metrics_task = tokio::task::spawn({
let registry = registry.clone();
async move {
let addr = "0.0.0.0:9100".parse().unwrap();
start_metrics_server(addr, registry).await
}
});


// Endpoint 메트릭스 등록
let endpoint = Endpoint::builder().bind().await?;
registry.write().unwrap().register_all(endpoint.metrics());
```

1.5 주요 성능 지표 해석

연결 품질 지표

1. **직접 연결 비율**: `num_direct_conns_added / (num_direct_conns_added + num_relay_conns_added)`
   - 높을수록 좋음 (낮은 지연시간, 높은 처리량)
   - 목표: 80% 이상
2. **연결 성공률**: `connection_handshake_success / nodes_contacted`
   - 네트워크 연결성 품질 지표
   - 목표: 95% 이상

3. **릴레이 의존도**: `recv_data_relay / (recv_data_ipv4 + recv_data_ipv6 + recv_data_relay)`

- 낮을수록 좋음 (직접 연결 선호)
- 목표: 20% 미만

네트워크 성능 지표

1. **패킷 드롭률**: `*_dropped / (*_sent + *_dropped)`

- 네트워크 혼잡 또는 오류 지표
- 목표: 1% 미만

2. **Discovery 효율성**: `recv_disco_pong / sent_disco_ping`

- 노드 발견 성공률
- 목표: 80% 이상

3. **GRO 활용률**: `recv_gro_datagrams / recv_datagrams`

- 네트워크 최적화 지표
- 높을수록 좋음

1.6 메트릭스 데이터 활용 방안

알림 설정

중요 임계값:

- 직접 연결 비율 < 50%: 네트워크 연결성 문제
- 패킷 드롭률 > 5%: 네트워크 혼잡 또는 오류
- 릴레이 서버 연결 실패 > 10%: 릴레이 서버 문제
- 핸드셰이크 실패율 > 10%: 인증 또는 네트워크 문제

대시보드 구성

핵심 메트릭스 대시보드:

1. 연결 상태 (직접/릴레이 연결 수)
2. 데이터 전송량 (송신/수신 바이트)
3. 네트워크 품질 (지연시간, 패킷 손실)
4. 오류율 (연결 실패, 패킷 드롭)

용량 계획

메트릭스 기반 확장 지표:

- 동시 연결 수 증가 추세
- 데이터 전송량 증가 패턴
- 릴레이 서버 부하 분산 필요성
- 네트워크 대역폭 사용량

2. 설정 관리 및 튜닝 가이드

2.1 Endpoint 빌더 설정 옵션

iroh의 `Endpoint::builder()` 는 다양한 설정 옵션을 제공하여 네트워크 환경과 애플리케이션 요구사항에 맞게 최적화할 수 있습니다.

기본 네트워크 설정

바인딩 주소 설정:

```
let  endpoint = Endpoint::builder()
.bind_addr_v4("0.0.0.0:8080".parse()?) // IPv4 바인딩 주소
.bind_addr_v6("[::]:8080".parse()?) // IPv6 바인딩 주소
.bind()
```

```
.await?;
```

권장 설정:

- 개발 환경: `0.0.0.0:0` (랜덤 포트)
- 프로덕션 환경: 고정 포트 사용 (방화벽 설정 용이)
- IPv6 지원 환경: 듀얼 스택 설정 권장

인증 및 보안 설정

Secret Key 관리:

```
// 새로운 키 생성 (개발/테스트)
let endpoint = Endpoint::builder()
.secret_key(SecretKey::generate(rand::rngs::OsRng))
.bind()
.await?;

// 기존 키 사용 (프로덕션)
let secret_key = SecretKey::from_bytes(&stored_key_bytes)?;
let endpoint = Endpoint::builder()
.secret_key(secret_key)
.bind()
.await?;
```

ALPN 프로토콜 설정:

```
let endpoint = Endpoint::builder()
.alpns(vec![
b"myapp/1.0".to_vec(),
b"myapp/0.9".to_vec(), // 하위 호환성
])
.bind()
.await?;
```

릴레이 서버 설정

기본 릴레이 사용:

```
let endpoint = Endpoint::builder()
.relay_mode(RelayMode::Default) // n0.computer 릴레이 사용
.bind()
.await?;
```

커스텀 릴레이 설정:

```
use iroh_relay::{RelayMap, RelayNode, RelayQuicConfig};

let relay_map = RelayMap::from_iter([
RelayNode {
url: "https://my-relay.example.com".parse()?.into(),
quic: Some(RelayQuicConfig::default()),
```

```
}
]);

let endpoint = Endpoint::builder()
.relay_mode(RelayMode::Custom(relay_map))
.bind()
.await?;
```

릴레이 비활성화:

```
let endpoint = Endpoint::builder()
.relay_mode(RelayMode::Disable) // 직접 연결만 사용
.bind()
.await?;
```

2.2 Discovery 서비스 설정

DNS Discovery 설정

기본 n0 DNS 사용:

```
let endpoint = Endpoint::builder()
.discovery_n0() // n0.computer DNS 서비스 사용
.bind()
.await?;
```

커스텀 DNS Discovery:

```
use iroh::discovery::dns::DnsDiscovery;

let  endpoint = Endpoint::builder()
.add_discovery(DnsDiscovery::new("my-dns.example.com".to_string()))
.bind()
.await?;
```

DHT Discovery 설정 (선택적)

```
#[cfg(feature = "discovery-pkarr-dht")]
let  endpoint = Endpoint::builder()
.discovery_dht() // Mainline DHT 사용
.bind()
.await?;
```

로컬 네트워크 Discovery

```
#[cfg(feature = "discovery-local-network")]
let  endpoint = Endpoint::builder()
.discovery_local_network() // mDNS 사용
.bind()
.await?;
```

다중 Discovery 서비스

```rust
let endpoint = Endpoint::builder()
.discovery_n0()
.discovery_local_network()
.discovery_dht()
.bind()
.await?;
```

2.3 QUIC Transport 설정

기본 Transport Config 튜닝

```rust
use quinn::TransportConfig;
use std::time::Duration;

let mut transport_config = TransportConfig::default();

// Keep-alive 설정
transport_config.keep_alive_interval(Some(Duration::from_secs(5)));

// 동시 스트림 수 제한
transport_config.max_concurrent_bidi_streams(100u32.into());
transport_config.max_concurrent_uni_streams(100u32.into());
```

```
// 데이터그램 크기 설정
transport_config.max_datagram_size(Some(1200));

// 유휴 타임아웃
transport_config.max_idle_timeout(Some(Duration::from_secs(30).try_into()?));

let endpoint = Endpoint::builder()
.transport_config(transport_config)
.bind()
.await?;
```

네트워크 환경별 최적화

고지연 네트워크 (위성, 국제 연결):

```
let mut transport_config = TransportConfig::default();
transport_config.max_idle_timeout(Some(Duration::from_secs(60).try_into()?));
transport_config.keep_alive_interval(Some(Duration::from_secs(10)));
// 더 큰 초기 윈도우 크기
transport_config.initial_rtt(Duration::from_millis(500));
```

고대역폭 네트워크:

```
let mut transport_config = TransportConfig::default();
```

```
transport_config.max_concurrent_bidi_streams(1000u32.into());
transport_config.max_concurrent_uni_streams(1000u32.into());
transport_config.receive_window(8 * 1024 * 1024); // 8MB
transport_config.send_window(8 * 1024 * 1024); // 8MB
```

모바일/불안정 네트워크:

```
let mut transport_config = TransportConfig::default();
transport_config.keep_alive_interval(Some(Duration::from_secs(2)));
transport_config.max_idle_timeout(Some(Duration::from_secs(15).try_into()?));
// 더 빠른 재전송
transport_config.initial_rtt(Duration::from_millis(100));
```

2.4 고급 설정 옵션

DNS Resolver 설정

```
#[cfg(not(wasm_browser))]
use crate::dns::DnsResolver;

let dns_resolver = DnsResolver::new()?;
let endpoint = Endpoint::builder()
.dns_resolver(dns_resolver)
.bind()
```

```
.await?;
```

프록시 설정

```
// 환경변수에서 프록시 설정 읽기
let endpoint = Endpoint::builder()
.proxy_from_env()
.bind()
.await?;

// 직접 프록시 URL 설정
let endpoint = Endpoint::builder()
.proxy_url("http://proxy.example.com:8080".parse()?)
.bind()
.await?;
```

TLS 키 로깅 (디버깅용)

```
// SSLKEYLOGFILE 환경변수 설정 시 TLS 키 로깅
let endpoint = Endpoint::builder()
.keylog(true)
.bind()
.await?;
```

2.5 환경별 설정 권장사항

개발 환경

```
let endpoint = Endpoint::builder()
.bind_addr_v4("127.0.0.1:0".parse()?) // 로컬호스트, 랜덤 포트
.relay_mode(RelayMode::Default) // 기본 릴레이 사용
.discovery_n0() // 기본 discovery
.keylog(true) // 디버깅용 키 로깅
.bind()
.await?;
```

테스트 환경

```
let endpoint = Endpoint::builder()
.relay_mode(RelayMode::Disable) // 릴레이 비활성화 (빠른 테스트)
.discovery_local_network() // 로컬 네트워크만 사용
.transport_config({
let mut config = TransportConfig::default();
config.max_idle_timeout(Some(Duration::from_secs(5).try_into()?));
config
})
.bind()
.await?;
```

프로덕션 환경

```
let endpoint = Endpoint::builder()
.bind_addr_v4("0.0.0.0:7842".parse()?) // 고정 포트
.bind_addr_v6("[::]:7842".parse()?) // 듀얼 스택
.secret_key(load_persistent_key()?) // 영구 키 사용
.relay_mode(RelayMode::Custom(production_relay_map()))
.discovery_n0()
.transport_config({
let mut config = TransportConfig::default();
config.keep_alive_interval(Some(Duration::from_secs(5)));
config.max_idle_timeout(Some(Duration::from_secs(30).try_into()?));
config.max_concurrent_bidi_streams(500u32.into());
config
})
.bind()
.await?;
```

2.6 설정 검증 및 모니터링

설정 유효성 검사

```
// Endpoint 생성 후 설정 확인
let endpoint = Endpoint::builder().bind().await?;
```

```
// 바인딩된 소켓 주소 확인
let bound_sockets = endpoint.bound_sockets();
println!("바인딩된 소켓: {:?}", bound_sockets);

// 노드 ID 확인
let node_id = endpoint.node_id();
println!("노드 ID: {}", node_id);

// 릴레이 서버 확인
let relay_url = endpoint.home_relay().initialized().await;
println!("홈 릴레이: {}", relay_url);
```

런타임 설정 모니터링

```
// 직접 주소 변경 모니터링
let mut direct_addrs_watcher = endpoint.direct_addresses();
tokio::spawn(async move {
while let Some(addrs) = direct_addrs_watcher.next().await {
if let Some(addrs) = addrs {
println!("직접 주소 업데이트: {:?}", addrs);
}
}
});
```

```
// 네트워크 보고서 모니터링
let mut net_report_watcher = endpoint.net_report();
tokio::spawn(async move {
while let Some(report) = net_report_watcher.next().await {
if let Some(report) = report {
println!("네트워크 보고서: {:?}", report);
}
}
});
```##

3. 트러블슈팅 가이드

3.1 일반적인 연결 문제

연결 실패 문제 진단

1. 기본 연결 실패 (`ConnectError`)

```rust
match endpoint.connect(node_addr, ALPN).await {
Err(ConnectError::Connect { source }) => {
match source.as_ref() {
```

```
ConnectWithOptsError::NoAddress { .. } => {
println!("주소 정보 없음 - Discovery 서비스 확인 필요");
}
ConnectWithOptsError::SelfConnect { .. } => {
println!("자기 자신에게 연결 시도 - 노드 ID 확인");
}
ConnectWithOptsError::Quinn { source } => {
println!("QUIC 연결 실패: {}", source);
}
_ => println!("기타 연결 오류: {}", source),
}
}
Err(ConnectError::Connection { source }) => {
println!("연결 후 오류: {}", source);
}
Ok(conn) => {
println!("연결 성공");
}
}
```

2. 바인딩 실패 (`BindError`)

```
match Endpoint::builder().bind().await {
```

```
Err(BindError::MagicSpawn { source }) => {
match  source {
CreateHandleError::BindSockets { source } => {
println!("소켓 바인딩 실패: {} - 포트 사용 중이거나 권한 부족", source);
}
CreateHandleError::CreateQuinnEndpoint { source } => {
println!("QUIC 엔드포인트 생성 실패: {}", source);
}
CreateHandleError::CreateNetmonMonitor { source } => {
println!("네트워크 모니터 생성 실패: {}", source);
}
_ => println!("기타 MagicSock 생성 오류: {}", source),
}
}
Err(BindError::Discovery { source }) => {
println!("Discovery 서비스 초기화 실패: {}", source);
}
Ok(endpoint) => {
println!("Endpoint 생성 성공");
}
}
```

연결 문제 해결 단계

1단계: 기본 네트워크 연결성 확인

```
// 네트워크 보고서 확인
let report = endpoint.net_report().initialized().await;
if let Some(report) = report {
println!("IPv4 가능: {}", report.ipv4);
println!("IPv6 가능: {}", report.ipv6);
println!("UDP 가능: {}", report.udp);
println!("ICMP 가능: {}", report.icmp_v4);
if !report.udp {
println!("경고: UDP 트래픽이 차단됨 - 방화벽 설정 확인");
}
}
```

2단계: 릴레이 서버 연결 확인

```
// 홈 릴레이 서버 확인
let home_relay = endpoint.home_relay().initialized().await;
println!("홈 릴레이: {}", home_relay);

// 릴레이 연결 상태 확인
let metrics = endpoint.metrics();
let relay_conns = metrics.magicsock.num_relay_conns_added.get();
if relay_conns == 0 {
```

```
println!("경고: 릴레이 서버에 연결되지 않음");
}
```

3단계: Discovery 서비스 확인

```
// Discovery 스트림 모니터링
let mut discovery_stream = endpoint.discovery();
tokio::spawn(async move {
while let Some(discovery_item) = discovery_stream.next().await {
match discovery_item {
Ok(item) => println!("노드 발견: {:?}", item),
Err(e) => println!("Discovery 오류: {}", e),
}
}
});
```

3.2 네트워크 환경별 문제 해결

NAT/방화벽 환경

문제 증상:

- 직접 연결 실패, 릴레이만 사용
- `num_direct_conns_added` 메트릭이 0
- 높은 지연시간

해결 방법:

1. **UPnP/NAT-PMP 활성화 확인**

```
// 포트 매핑 메트릭스 확인 (non-WASM 환경)
#[cfg(not(wasm_browser))]
{
let portmap_metrics = &endpoint.metrics().portmapper;
println!("포트 매핑 시도: {}", portmap_metrics.mapping_requests.get());
println!("포트 매핑 성공: {}", portmap_metrics.mapping_success.get());
}
```

2. **STUN 서버 응답 확인**

```
let metrics = endpoint.metrics();
let disco_pings = metrics.magicsock.sent_disco_ping.get();
let disco_pongs = metrics.magicsock.recv_disco_pong.get();
let success_rate = if disco_pings > 0 {
(disco_pongs as f64 / disco_pings as f64) * 100.0
} else {
0.0
};
println!("Discovery 성공률: {:.1}%", success_rate);
```

3. **방화벽 규칙 설정**

```
# Linux iptables 예시
```

```
sudo iptables -A INPUT -p udp --dport 7842 -j ACCEPT
sudo iptables -A OUTPUT -p udp --sport 7842 -j ACCEPT

# Windows Firewall 예시
netsh advfirewall firewall add rule name="iroh UDP" dir=in action=allow
```

기업 네트워크 환경

문제 증상:

- 모든 연결이 릴레이를 통해서만 가능
- Discovery 서비스 실패
- 프록시 관련 오류

해결 방법:

1. **프록시 설정**

```
let endpoint = Endpoint::builder()
.proxy_from_env() // 환경변수에서 프록시 설정 읽기
.bind()
.await?;

// 또는 직접 설정
let endpoint = Endpoint::builder()
.proxy_url("http://corporate-proxy:8080".parse()?)
.bind()
```

```
.await?;
```

2. **DNS 설정**

```
#[cfg(not(wasm_browser))]
{
use crate::dns::DnsResolver;
// 커스텀 DNS 서버 사용
let dns_resolver = DnsResolver::new_with_servers(vec![
"8.8.8.8:53".parse()?,
"1.1.1.1:53".parse()?,
])?;
let endpoint = Endpoint::builder()
.dns_resolver(dns_resolver)
.bind()
.await?;
}
```

모바일/불안정 네트워크

문제 증상:

- 빈번한 연결 끊김
- 높은 패킷 손실률
- 네트워크 전환 시 연결 실패

해결 방법:

1. **Keep-alive 간격 단축**

```
let mut transport_config = TransportConfig::default();
transport_config.keep_alive_interval(Some(Duration::from_secs(2)));
transport_config.max_idle_timeout(Some(Duration::from_secs(10).try_into()?));

let endpoint = Endpoint::builder()
.transport_config(transport_config)
.bind()
.await?;
```

2. **네트워크 변경 알림**

```
// 네트워크 변경 시 수동으로 알림
endpoint.notify_network_change();
```

3.3 로그 분석 및 디버깅

로그 레벨 설정

```
// 환경변수로 설정
// RUST_LOG=iroh=debug,iroh::magicsock=trace

// 코드에서 설정
```

```rust
use tracing_subscriber::{EnvFilter, fmt};


tracing_subscriber::fmt()
.with_env_filter(
EnvFilter::from_default_env()
.add_directive("iroh=debug".parse()?)
.add_directive("iroh::magicsock=trace".parse()?)
)
.init();
```

주요 로그 패턴 분석

1. 연결 설정 로그

```
DEBUG iroh::endpoint: new connection from <node_id> with ALPN <protocol>
TRACE iroh::magicsock: direct connection established to <node_id>
INFO iroh::magicsock: connection became direct for <node_id>
```

2. Discovery 로그

```
DEBUG iroh::discovery: resolved node <node_id> to <addresses>
WARN iroh::discovery: discovery failed for <node_id>: <error>
TRACE iroh::magicsock: sent disco ping to <node_id>
```

3. 릴레이 관련 로그

```
INFO iroh::magicsock: connected to relay <relay_url>

WARN iroh::magicsock: relay connection failed: <error>

DEBUG iroh::magicsock: using relay for <node_id>
```

TLS 키 로깅 (디버깅용)

```
// SSLKEYLOGFILE 환경변수 설정
std::env::set_var("SSLKEYLOGFILE", "/tmp/iroh-keys.log");


let  endpoint = Endpoint::builder()
.keylog(true) // TLS 키 로깅 활성화
.bind()
.await?;
```

3.4 성능 문제 진단

지연시간 문제

진단 방법:

```
// 연결별 통계 확인
let  conn_stats = connection.stats();
println!("RTT: {:?}", conn_stats.path.rtt);
println!("패킷 손실: {}", conn_stats.path.lost_packets);
```

```
println!("혼잡 윈도우: {}", conn_stats.path.cwnd);
```

해결 방법:

1. 직접 연결 비율 개선
2. 더 가까운 릴레이 서버 사용
3. QUIC 혼잡 제어 알고리즘 조정

처리량 문제

진단 방법:

```
let metrics = endpoint.metrics();
let sent_bytes = metrics.magicsock.send_data.get();
let recv_bytes = metrics.magicsock.recv_data_ipv4.get() +
metrics.magicsock.recv_data_ipv6.get();
println!("송신: {} 패킷, 수신: {} 패킷", sent_bytes, recv_bytes);
```

해결 방법:

1. 동시 스트림 수 증가
2. 수신/송신 윈도우 크기 조정
3. 데이터그램 크기 최적화

3.5 일반적인 오류 메시지와 해결책

"No addressing information available"

원인: Discovery 서비스가 설정되지 않았거나 실패

해결책:

```
let endpoint = Endpoint::builder()
.discovery_n0() // 기본 discovery 서비스 추가
.bind()
.await?;
```

"Failed to create bind sockets"

원인: 포트가 이미 사용 중이거나 권한 부족

해결책:

```
let endpoint = Endpoint::builder()
.bind_addr_v4("0.0.0.0:0".parse()?) // 랜덤 포트 사용
.bind()
.await?;
```

"Connecting to ourself is not supported"

원인: 자기 자신의 노드 ID로 연결 시도

해결책: 대상 노드 ID가 올바른지 확인

"Discovery service failed"

원인: DNS 해상도 실패 또는 네트워크 연결 문제

해결책:

```
// 여러 discovery 서비스 사용
let endpoint = Endpoint::builder()
```

```
.discovery_n0()
.discovery_local_network()
.bind()
.await?;
```

3.6 디버깅 도구 및 유틸리티

네트워크 상태 모니터링

```
async fn monitor_network_status(endpoint: &Endpoint) {
let mut net_report_watcher = endpoint.net_report();
while let Some(report) = net_report_watcher.next().await {
if let Some(report) = report {
println!("=== 네트워크 상태 ===");
println!("IPv4: {}, IPv6: {}", report.ipv4, report.ipv6);
println!("UDP: {}, ICMP: {}", report.udp, report.icmp_v4);
if let Some(portmap) = &report.portmap {
println!("포트 매핑: {:?}", portmap);
}
for (relay_url, latency) in &report.relay_latency {
println!("릴레이 {}: {}ms", relay_url, latency.as_millis());
}
}
}
```

```
}
```

연결 상태 대시보드

```
async fn connection_dashboard(endpoint: &Endpoint) {
let metrics = endpoint.metrics();
println!("=== 연결 통계 ===");
println!("직접 연결: {}", metrics.magicsock.num_direct_conns_added.get());
println!("릴레이 연결: {}", metrics.magicsock.num_relay_conns_added.get());
println!("핸드셰이크 성공: {}", metrics.magicsock.connection_handshake_success.get
println!("\n=== 데이터 전송 ===");
println!("송신 패킷: {}", metrics.magicsock.send_data.get());
println!("수신 패킷 (IPv4): {}", metrics.magicsock.recv_data_ipv4.get());
println!("수신 패킷 (IPv6): {}", metrics.magicsock.recv_data_ipv6.get());
println!("릴레이 수신: {}", metrics.magicsock.recv_data_relay.get());
println!("\n=== Discovery ===");
println!("Ping 송신: {}", metrics.magicsock.sent_disco_ping.get());
println!("Pong 수신: {}", metrics.magicsock.recv_disco_pong.get());
println!("잘못된 키: {}", metrics.magicsock.recv_disco_bad_key.get());
}
```## 4. 성능 최

적화 가이드

4.1 성능에 영향을 미치는 주요 요소

네트워크 연결 유형

직접 연결 vs 릴레이 연결

- 직접 연결: 낮은 지연시간, 높은 처리량
- 릴레이 연결: 높은 지연시간, 제한된 처리량

최적화 목표: 직접 연결 비율 최대화

```rust
// 직접 연결 비율 모니터링
let metrics = endpoint.metrics();
let direct_conns = metrics.magicsock.num_direct_conns_added.get();
let relay_conns = metrics.magicsock.num_relay_conns_added.get();
let total_conns = direct_conns + relay_conns;

if total_conns > 0 {
let direct_ratio = (direct_conns as f64 / total_conns as f64) * 100.0;
println!("직접 연결 비율: {:.1}%", direct_ratio);
if direct_ratio < 80.0 {
println!("경고: 직접 연결 비율이 낮음 - NAT/방화벽 설정 확인 필요");
}
```

```
}
```

QUIC 전송 매개변수

혼잡 제어 알고리즘

- 기본값: Cubic
- 고지연 네트워크: BBR 고려

윈도우 크기

- 수신 윈도우: 메모리 사용량과 처리량의 균형
- 송신 윈도우: 네트워크 대역폭 활용도

4.2 메모리 사용량 최적화

연결 풀 관리

```
use std::collections::HashMap;
use std::sync::Arc;
use tokio::sync::RwLock;

struct ConnectionPool {
connections: Arc<RwLock<HashMap<NodeId, quinn::Connection>>>,
max_connections: usize,
}

impl ConnectionPool {
fn  new(max_connections: usize) -> Self {
```

```
Self {
connections: Arc::new(RwLock::new(HashMap::new())),
max_connections,
}
}
async fn get_or_create(&self, endpoint: &Endpoint, node_addr: NodeAddr, alpn:
-> Result<quinn::Connection, ConnectError> {
let mut connections = self.connections.write().await;
// 기존 연결 재사용
if let Some(conn) = connections.get(&node_addr.node_id) {
if !conn.close_reason().is_some() {
return Ok(conn.clone());
} else {
connections.remove(&node_addr.node_id);
}
}
// 연결 수 제한
if connections.len() >= self.max_connections {
// LRU 방식으로 오래된 연결 제거
if let Some(oldest_node_id) = self.find_oldest_connection(&connections).await {
connections.remove(&oldest_node_id);
}
}
```

```
// 새 연결 생성
let  conn = endpoint.connect(node_addr.clone(), alpn).await?;
connections.insert(node_addr.node_id, conn.clone());
Ok(conn)
}
async  fn  find_oldest_connection(&self, connections: &HashMap<NodeId, quinn::Co
// 구현: 가장 오래된 연결 찾기
connections.keys().next().copied()
}
}
```

버퍼 크기 최적화

```
let  mut  transport_config = TransportConfig::default();

// 메모리 제한 환경
transport_config.receive_window(1024 * 1024); // 1MB
transport_config.send_window(1024 * 1024); // 1MB
transport_config.max_concurrent_bidi_streams(50u32.into());
transport_config.max_concurrent_uni_streams(50u32.into());

// 고성능 환경
transport_config.receive_window(16 * 1024 * 1024); // 16MB
```

```
transport_config.send_window(16 * 1024 * 1024); // 16MB
transport_config.max_concurrent_bidi_streams(1000u32.into());
transport_config.max_concurrent_uni_streams(1000u32.into());
```

4.3 CPU 사용량 최적화

비동기 작업 최적화

```
use tokio::task::JoinSet;

async fn handle_multiple_connections(endpoint: &Endpoint, node_addrs: Vec<Node
let mut join_set = JoinSet::new();
// 동시 연결 수 제한
let semaphore = Arc::new(tokio::sync::Semaphore::new(10));
for node_addr in node_addrs {
let endpoint = endpoint.clone();
let semaphore = semaphore.clone();
join_set.spawn(async move {
let _permit = semaphore.acquire().await.unwrap();
match endpoint.connect(node_addr, b"myapp/1.0").await {
Ok(conn) => {
// 연결 처리
handle_connection(conn).await;
}
```

```
Err(e) => {
eprintln!("연결 실패: {}", e);
}
}
});
}
// 모든 작업 완료 대기
while let Some(result) = join_set.join_next().await {
if let Err(e) = result {
eprintln!("작업 실패: {}", e);
}
}
}

async fn handle_connection(conn: quinn::Connection) {
// 연결별 처리 로직
}
```

스레드 풀 최적화

```
// Tokio 런타임 설정
let rt = tokio::runtime::Builder::new_multi_thread()
.worker_threads(num_cpus::get()) // CPU 코어 수만큼 워커 스레드
```

```
.thread_name("iroh-worker")
.thread_stack_size(2 * 1024 * 1024) // 2MB 스택 크기
.enable_all()
.build()?;

rt.block_on(async {
let  endpoint = Endpoint::builder().bind().await?;
// 애플리케이션 로직
});
```

4.4 네트워크 대역폭 효율성 개선

데이터그램 크기 최적화

```
let  mut  transport_config = TransportConfig::default();

// MTU 기반 최적화 (일반적으로 1500바이트 이더넷)
transport_config.max_datagram_size(Some(1200)); // 헤더 오버헤드 고려

// 점보 프레임 환경 (10Gbps 네트워크)
transport_config.max_datagram_size(Some(8192));
```

압축 및 직렬화 최적화

```
use serde::{Serialize, Deserialize};
```

```rust
use postcard; // 효율적인 바이너리 직렬화


#[derive(Serialize, Deserialize)]
struct Message {
id: u64,
data: Vec<u8>,
timestamp: u64,
}


async fn send_optimized_message(mut send: quinn::SendStream, message: &Messag
// 바이너리 직렬화 (JSON보다 효율적)
let serialized = postcard::to_stdvec(message)?;
// 압축 (선택적)
#[cfg(feature = "compression")]
let compressed = compress_data(&serialized)?;
#[cfg(not(feature = "compression"))]
let compressed = serialized;
// 길이 프리픽스 추가
let len = compressed.len() as u32;
send.write_all(&len.to_be_bytes()).await?;
send.write_all(&compressed).await?;
Ok(())
}
```

```rust
#[cfg(feature = "compression")]
fn compress_data(data: &[u8]) -> Result<Vec<u8>, Box<dyn std::error::Error>> {
use flate2::{Compression, write::GzEncoder};
use std::io::Write;
let mut encoder = GzEncoder::new(Vec::new(), Compression::fast());
encoder.write_all(data)?;
Ok(encoder.finish()?)
}
```

배치 처리

```rust
use tokio::time::{interval, Duration};

struct BatchSender {
send_stream: quinn::SendStream,
batch: Vec<Message>,
batch_size: usize,
flush_interval: Duration,
}

impl BatchSender {
fn new(send_stream: quinn::SendStream, batch_size: usize) -> Self {
```

```
Self {
send_stream,
batch: Vec::with_capacity(batch_size),
batch_size,
flush_interval: Duration::from_millis(10),
}
}
async fn send_message(&mut self, message: Message) -> Result<(), Box<dyn std
self.batch.push(message);
if self.batch.len() >= self.batch_size {
self.flush().await?;
}
Ok(())
}
async fn flush(&mut self) -> Result<(), Box<dyn std::error::Error>> {
if self.batch.is_empty() {
return Ok(());
}
let serialized = postcard::to_stdvec(&self.batch)?;
let len = serialized.len() as u32;
self.send_stream.write_all(&len.to_be_bytes()).await?;
self.send_stream.write_all(&serialized).await?;
self.batch.clear();
```

```
Ok(())
}
async fn run_with_timer(&mut self) {
let mut timer = interval(self.flush_interval);
loop {
timer.tick().await;
if let Err(e) = self.flush().await {
eprintln!("배치 플러시 실패: {}", e);
break;
}
}
}
}
```

4.5 Discovery 성능 최적화

캐싱 전략

```
use std::collections::HashMap;
use std::time::{Duration, Instant};

struct NodeAddrCache {
cache: HashMap<NodeId, (NodeAddr, Instant)>,
```

```
ttl: Duration,
}


impl NodeAddrCache {
fn  new(ttl: Duration) -> Self {
Self {
cache: HashMap::new(),
ttl,
}
}
fn  get(&mut  self, node_id: &NodeId) -> Option<NodeAddr> {
if  let Some((addr, timestamp)) = self.cache.get(node_id) {
if  timestamp.elapsed() < self.ttl {
return Some(addr.clone());
} else {
self.cache.remove(node_id);
}
}
None
}
fn  insert(&mut  self, node_id: NodeId, addr: NodeAddr) {
self.cache.insert(node_id, (addr, Instant::now()));
}
```

```
fn cleanup_expired(&mut self) {
let now = Instant::now();
self.cache.retain(|_, (_, timestamp)| now.duration_since(*timestamp) < self.ttl)
}
}
```

병렬 Discovery

```
async fn discover_nodes_parallel(
endpoint: &Endpoint,
node_ids: Vec<NodeId>,
) -> HashMap<NodeId, NodeAddr> {
let mut discovery_stream = endpoint.discovery();
let mut results = HashMap::new();
let mut pending = node_ids.into_iter().collect::<std::collections::HashSet<_>>
// 타임아웃 설정
let timeout = tokio::time::sleep(Duration::from_secs(10));
tokio::pin!(timeout);
loop {
tokio::select! {
discovery_item = discovery_stream.next() => {
if let Some(Ok(item)) = discovery_item {
if pending.remove(&item.node_id) {
results.insert(item.node_id, item.node_addr);
```

```
if pending.is_empty() {
break;
}
}
}
}
_ = &mut timeout => {
println!("Discovery 타임아웃: {} 노드 미발견", pending.len());
break;
}
}
}
results
}
```

4.6 릴레이 서버 최적화

릴레이 선택 최적화

```
async fn select_best_relay(endpoint: &Endpoint) -> Option<RelayUrl> {
let report = endpoint.net_report().initialized().await?;
// 지연시간 기준으로 최적 릴레이 선택
let best_relay = report.relay_latency
.iter()
```

```
.min_by_key(|(_, latency)| *latency)
.map(|(url, _)| url.clone());
if let Some(relay_url) = &best_relay {
println!("최적 릴레이 선택: {} ({}ms)",
relay_url,
report.relay_latency.get(relay_url)?.as_millis());
}
best_relay
}
```

커스텀 릴레이 맵 구성

```
use iroh_relay::{RelayMap, RelayNode, RelayQuicConfig};

fn create_optimized_relay_map() -> RelayMap {
RelayMap::from_iter([
// 지역별 릴레이 서버
RelayNode {
url: "https://asia-relay.example.com".parse().unwrap().into(),
quic: Some(RelayQuicConfig::default()),
},
RelayNode {
url: "https://europe-relay.example.com".parse().unwrap().into(),
```

```
quic: Some(RelayQuicConfig::default()),
},
RelayNode {
url: "https://us-relay.example.com".parse().unwrap().into(),
quic: Some(RelayQuicConfig::default()),
},
])
}
```

4.7 모니터링 기반 성능 튜닝

성능 메트릭스 수집

```
struct PerformanceMonitor {
endpoint: Endpoint,
metrics_history: Vec<PerformanceSnapshot>,
}

#[derive(Clone)]
struct PerformanceSnapshot {
timestamp: Instant,
direct_conn_ratio: f64,
avg_rtt: Duration,
throughput_mbps: f64,
```

```
packet_loss_rate: f64,
}


impl PerformanceMonitor {
async fn collect_snapshot(&mut self) -> PerformanceSnapshot {
let metrics = self.endpoint.metrics();
// 직접 연결 비율 계산
let direct_conns = metrics.magicsock.num_direct_conns_added.get();
let relay_conns = metrics.magicsock.num_relay_conns_added.get();
let total_conns = direct_conns + relay_conns;
let direct_conn_ratio = if total_conns > 0 {
direct_conns as f64 / total_conns as f64
} else {
0.0
};
// 처리량 계산 (간단한 예시)
let sent_packets = metrics.magicsock.send_data.get();
let recv_packets = metrics.magicsock.recv_data_ipv4.get() +
metrics.magicsock.recv_data_ipv6.get();
let total_packets = sent_packets + recv_packets;
PerformanceSnapshot {
timestamp: Instant::now(),
direct_conn_ratio,
```

```
avg_rtt: Duration::from_millis(50), // 실제로는 연결별 RTT 평균
throughput_mbps: total_packets  as f64 * 1.2 / 1024.0 / 1024.0, // 추정값
packet_loss_rate: 0.01, // 실제로는 패킷 손실률 계산
}
}
async  fn  analyze_performance(&self) -> PerformanceAnalysis {
let  recent_snapshots = self.metrics_history
.iter()
.rev()
.take(10)
.collect::<Vec<_>>();
let  avg_direct_ratio = recent_snapshots
.iter()
.map(|s| s.direct_conn_ratio)
.sum::<f64>() / recent_snapshots.len() as f64;
let  avg_rtt = recent_snapshots
.iter()
.map(|s| s.avg_rtt.as_millis())
.sum::<u128>() / recent_snapshots.len() as u128;
PerformanceAnalysis {
avg_direct_ratio,
avg_rtt_ms: avg_rtt,
recommendations: self.generate_recommendations(avg_direct_ratio, avg_rtt),
```

```
}

}

fn generate_recommendations(&self, direct_ratio: f64, avg_rtt_ms: u128) -> Vec<

let mut recommendations = Vec::new();

if direct_ratio < 0.7 {

recommendations.push("직접 연결 비율이 낮습니다. NAT/방화벽 설정을 확인하세요.".to_str

}

if avg_rtt_ms > 200 {

recommendations.push("평균 RTT가 높습니다. 더 가까운 릴레이 서버를 사용하세요.".to_str

}

recommendations

}

}


struct PerformanceAnalysis {

avg_direct_ratio: f64,

avg_rtt_ms: u128,

recommendations: Vec<String>,

}
```

자동 튜닝

```
struct AutoTuner {

endpoint: Endpoint,
```

```
current_config: TransportConfig,
}

impl AutoTuner {
async fn tune_based_on_metrics(&mut self) {
let metrics = self.endpoint.metrics();
let mut new_config = self.current_config.clone();
// 패킷 손실률 기반 윈도우 크기 조정
let sent_packets = metrics.magicsock.send_data.get();
let error_packets = metrics.magicsock.send_data_network_down.get();
if sent_packets > 0 {
let error_rate = error_packets as f64 / sent_packets as f64;
if error_rate > 0.05 {
// 오류율이 높으면 윈도우 크기 감소
let current_window = new_config.send_window();
new_config.send_window(current_window * 3 / 4);
println!("높은 오류율로 인해 송신 윈도우 감소: {}", current_window * 3 / 4);
} else if error_rate < 0.01 {
// 오류율이 낮으면 윈도우 크기 증가
let current_window = new_config.send_window();
new_config.send_window(current_window * 5 / 4);
println!("낮은 오류율로 인해 송신 윈도우 증가: {}", current_window * 5 / 4);
}
```

```
}

self.current_config = new_config;

// 실제로는 새로운 연결에 적용

}

}
```

이 포괄적인 운영 및 모니터링 가이드를 통해 iroh 기반 애플리케이션을 프로덕션 환경에서 효율적으로 운영하고 최적화할 수 있습니다. 메트릭스 모니터링, 설정 튜닝, 트러블슈팅, 그리고 성능 최적화의 모든 측면을 다루어 안정적이고 고성능의 P2P 네트워킹 시스템을 구축할 수 있습니다.